

way that you create an instance of each of these classes is different—you need to specify a different set of arguments. However, once created, all the remaining methods of both objects are exactly the same. That’s because the two classes inherit from a common base class, named `PygWidgetsButton` ([Figure 10-7](#)).

`PygWidgetsButton` is an abstract class. Client code is not supposed to create an instance of it, and attempting to do so will generate an error message.

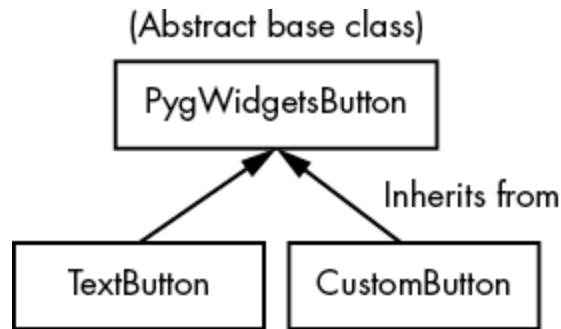


Figure 10-7: The `pygwidgets` `TextButton` and `CustomButton` classes both inherit from `PygWidgetsButton`.

Instead, `PygWidgetsButton` is subclassed by the `TextButton` and `CustomButton` classes. Those classes each provide the single method, `__init__()`, which will do whatever is needed to initialize their type of button. Each will then pass on identical arguments to the `__init__()` method of the base class,

`PygWidgetsButton`.

The `TextButton` class is used to build a text-based button with minimal art. This is helpful when trying to get programs up and running quickly. Here is the interface for creating a `TextButton` object:

```
def __init__(self, window, loc, text, width=None, height=40,
             textColor=PYGWIDGETS_BLACK,
             upColor= PYGWIDGETS_NORMAL_GRAY,
             overColor= PYGWIDGETS_OVER_GRAY,
             downColor=PYGWIDGETS_DOWN_GRAY,
             fontName=None, fontSize=20, soundOnClick=None,
             enterToActivate=False, callBack=None, nickname=None)
```

While many of the parameters default to reasonable values, the caller must provide a value for `text`, which will appear on the button. The